

**Evidencia de conocimiento: GA8-220501096-AA2-EV01 taller acerca de
integración, tecnologías emergentes y disruptivas**

ANÁLISIS Y DESARROLLO DE SOFTWARE

Ficha (2977481)

Autor(es): Hader Borré

Dirigido a:
Milton Barbosa

29/09/2025

Introducción

El desarrollo de aplicaciones móviles para Android se ha consolidado como una de las áreas más importantes de la tecnología actual, debido a la gran penetración de este sistema operativo en el mercado de dispositivos móviles. El presente taller busca brindar una comprensión básica sobre el ecosistema Android, sus componentes principales y las herramientas necesarias para el desarrollo de aplicaciones. Asimismo, se presenta la creación de una aplicación práctica llamada **Nexa Support**, diseñada en **MIT App Inventor**, con el fin de aplicar los conceptos investigados.

Objetivo General

Investigar y aplicar los conceptos básicos del desarrollo de aplicaciones móviles para Android, mediante la construcción de una aplicación funcional en MIT App Inventor.

Objetivos Específicos

1. Identificar los elementos fundamentales del sistema Android y su ecosistema de desarrollo.
2. Conocer el rol de los archivos APK, el Android SDK y los lenguajes de programación asociados.
3. Reconocer los principales entornos de desarrollo (IDEs) disponibles para crear aplicaciones móviles.
4. Comprender los conceptos de Android multiusuario y principio de mínimo privilegio en seguridad.
5. Construir una aplicación básica en MIT App Inventor aplicando interfaz gráfica, lógica de bloques y generación de un archivo APK.
6. Probar la aplicación en un dispositivo Android, evaluando su funcionamiento.

Sección 1 Taller

Investigación sobre el Desarrollo de Aplicaciones Móviles para Android

¿Qué es Android?

Android es un sistema operativo móvil desarrollado por **Google**, basado en el núcleo de **Linux** y diseñado principalmente para dispositivos móviles como teléfonos inteligentes y tabletas. Su principal característica es ser **código abierto**, lo que permite que fabricantes y desarrolladores adapten el sistema según sus necesidades. Actualmente, es el sistema operativo más usado en dispositivos móviles a nivel mundial.

Definición del concepto de APK

Un **APK (Android Package Kit)** es el formato de archivo que utilizan los dispositivos Android para distribuir e instalar aplicaciones. Es el equivalente a un archivo **.exe** en Windows o **.dmg** en macOS. Contiene todos los elementos necesarios para que la aplicación funcione: código compilado, recursos, imágenes, librerías y el archivo de manifiesto.

¿Qué es el Android SDK?

El **Android SDK (Software Development Kit)** es el conjunto de herramientas que proporciona Google para que los desarrolladores puedan crear aplicaciones para Android. Incluye:

- Compiladores.
- Emuladores de dispositivos.
- Bibliotecas de funciones.
- Documentación y ejemplos de código.
- Herramientas de depuración.

En conjunto, el SDK facilita la creación, prueba y optimización de las aplicaciones antes de publicarlas en la Google Play Store.

¿Cuál es el lenguaje utilizado para desarrollar aplicaciones para Android?

Históricamente, el principal lenguaje fue **Java**. Hoy en día, Google recomienda usar **Kotlin**, un lenguaje moderno, conciso y seguro que es 100% interoperable con Java. Además, en algunos casos también se utilizan otros lenguajes y frameworks:

- **C/C++** (con el NDK).
- **Dart** (con Flutter).
- **JavaScript/TypeScript** (con React Native).

¿Qué IDEs de desarrollo existen para codificar?

Los IDEs más usados en el desarrollo Android son:

1. **Android Studio** IDE oficial recomendado por Google, basado en IntelliJ IDEA.
2. **Eclipse (con ADT plugin)** se usaba antes, ahora en desuso.
3. **Visual Studio Code** para frameworks híbridos como Flutter o React Native.
4. **Xamarin (Visual Studio)** para desarrollar apps Android usando C# y .NET.
5. **Apache NetBeans** aunque no es el IDE oficial, permite programar en **Java** y puede integrarse con librerías y plugins para desarrollar aplicaciones Android. Muchos desarrolladores lo usan por su sencillez y porque es más liviano que Android Studio.

Android multiusuario

El **modo multiusuario** en Android es una funcionalidad que permite crear **varios perfiles de usuario en un mismo dispositivo**, similar a cómo funcionan las cuentas de usuario en un sistema operativo de escritorio. Cada usuario tiene su propio espacio independiente: aplicaciones, configuraciones, archivos y datos personales. Es útil para compartir un dispositivo sin comprometer la privacidad de cada persona.

Concepto de mínimo privilegio

El **principio de mínimo privilegio** establece que un usuario o aplicación debe tener **únicamente los permisos necesarios para realizar sus funciones**, y no más. En Android, esto se refleja en los permisos de las apps: por ejemplo, una aplicación de notas no debería pedir acceso a la cámara o al micrófono si no lo necesita. Esto mejora la seguridad y evita abusos de información.

Componentes esenciales de una aplicación Android

Una aplicación Android se compone principalmente de los siguientes elementos:

1. **Activities** → pantallas de la aplicación (interfaces gráficas).
2. **Services** → procesos que se ejecutan en segundo plano (ejemplo: música, descargas).
3. **Broadcast Receivers** → permiten que la app responda a mensajes o eventos del sistema (ejemplo: batería baja, llegada de un SMS).
4. **Content Providers** → manejan y comparten datos entre aplicaciones (ejemplo: lista de contactos).
5. **Manifest (AndroidManifest.xml)** → archivo central donde se definen permisos, actividades principales y configuraciones de la app.

Sección 2 Taller

Crear la primera aplicación Android.

Paso a Paso – Creación de la aplicación *Nexa Support* en MIT App Inventor

Paso 1. Acceso y creación del proyecto

1. Ingresé a MIT App Inventor.
2. Inicié sesión con mi cuenta de Google.
3. Fui al menú **Proyectos > Iniciar un nuevo proyecto**.
4. Asigné el nombre: **Nexa_Support**.

Este nombre identifica el proyecto dentro de App Inventor.

Paso 2. Configuración de la pantalla principal (*Screen1*)

1. MIT App Inventor crea por defecto una pantalla llamada **Screen1**.
2. En el panel derecho configuré:
 - **Título:** “Nexa support”.
 - **Color de la barra superior:** azul corporativo.
 - **Orientación:** vertical (para móviles Android).
 - **Visible:** verdadero.

Esta pantalla será la **pantalla de inicio de sesión** de la aplicación.

Paso 3: Creación de la interfaz gráfica

Desde la sección **Interfaz de usuario** arrastré y solté componentes dentro del simulador de teléfono:

1. Etiqueta de título

- Texto: **“Nexa support”**.
- Fuente: en negrita y centrada.

2. Etiqueta secundaria

- Texto: **“Soporte remoto eficiente y confiable”**.
- Función: servir como slogan.

3. Imagen

- Subí un recurso gráfico (fondo_web.png) con el robot y diseño tecnológico.
- Inserté el componente **Imagen** y lo vinculé al archivo.

4. Campos de texto

- Arrastré dos **CamposDeContraseña**.
- El primero para el **usuario/correo**.
- El segundo para la **contraseña**.
- Configuré Hint (texto de sugerencia): “Usuario” y “Contraseña”.

5. Interruptor

- Inserté un interruptor llamado **Mostrar_contraseña**.
- Permite alternar la visibilidad de la contraseña.

6. Botón de acceso

- Texto: **“Iniciar sesión”**.
- Color: azul con letras blancas.

7. Enlaces secundarios

- Texto 1: “¿No eres cliente aún? Regístrate”.
- Texto 2: “¿Olvidaste tu contraseña?”.
- Cada uno se configuró como **Etiquetas** con formato subrayado y color azul, simulando enlaces.

8. Pie de página

- Texto: “© 2025 Nexa Support. Todos los derechos reservados.”
- Ubicación: parte inferior de la pantalla.

Paso 4: Organización de los componentes

Para alinear bien los elementos utilicé **disposiciones horizontales y verticales** (Layout):

- Un **VerticalArrangement** principal, donde coloqué todos los elementos de arriba a abajo.
- Dentro de él, **HorizontalArrangement** para alinear los enlaces en la parte inferior.

Paso 5: Configuración de propiedades

En el panel derecho personalicé propiedades de cada componente:

- **Color de texto y de fondo.**
- **Tamaño de letra.**
- **Visibilidad inicial.**
- Para el campo de contraseña, marqué la casilla de “**Solo números**” como **falso**, permitiendo cualquier carácter.

Paso 6: Programación con bloques

1. Fui a la vista **Bloques**.
2. Creé la lógica básica del login:
 - Si el usuario y la contraseña coinciden con valores definidos (ejemplo: usuario: admin, contraseña: 12345), entonces mostrar un **Notificador** con el mensaje “**Bienvenido a Nexa Support**”.
 - En caso contrario, mostrar mensaje “**Credenciales incorrectas**”.
3. Configuré el bloque del interruptor **Mostrar_contraseña**:
 - Si está activado, el campo de contraseña se muestra en texto normal.
 - Si está desactivado, vuelve al formato oculto (asteriscos).

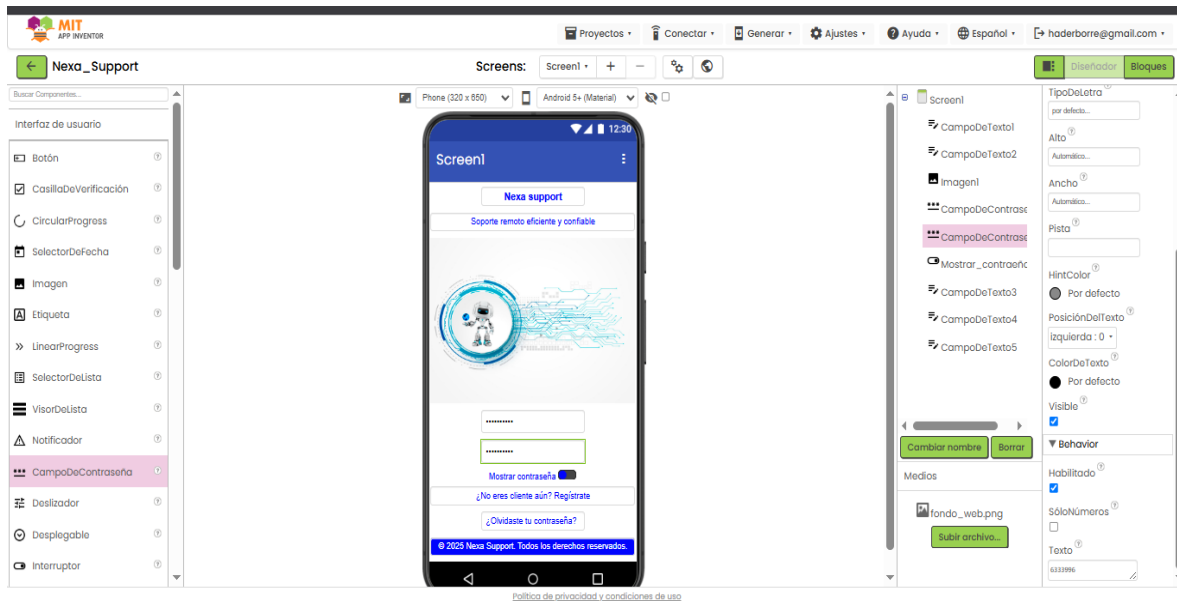
4. Vinculé los botones/enlaces secundarios a mensajes de alerta simulados (por ejemplo: “Función de registro en construcción”).

7. Pruebas en dispositivo/emulador

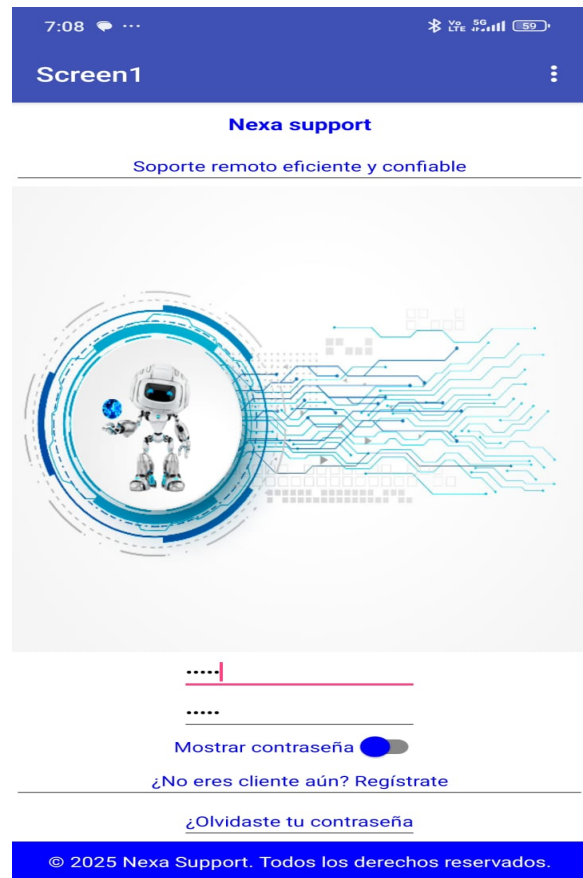
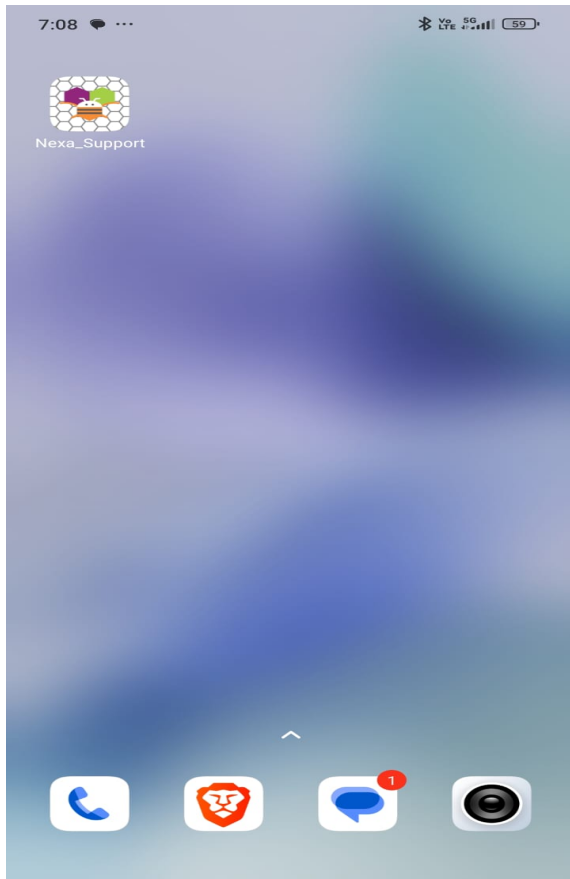
1. Desde el menú **Conectar > AI Companion**, escaneé el código QR con mi celular Android.
2. Probé en tiempo real el login, la visibilidad de contraseña y los botones de registro/recuperar clave.
3. Ajusté tamaños de texto y colores hasta que la interfaz quedó clara y legible.

8. Generación del APK

1. Una vez lista la aplicación, fui al menú **Generar > App (APK)**.
2. Guardé el archivo APK en mi computadora.
3. Lo instalé en mi dispositivo Android para comprobar el funcionamiento final.



Instalación en mi dispositivo nexa support



Conclusión

El taller permitió comprender los aspectos básicos del desarrollo de aplicaciones Android desde dos perspectivas: la **teórica**, con la investigación sobre Android, sus componentes y herramientas; y la **práctica**, con la creación de una aplicación real en **MIT App Inventor**. La construcción de **Nexa Support** permitió aplicar conceptos de interfaz gráfica, programación por bloques, validación de usuarios y empaquetado en formato APK para su instalación en un dispositivo Android. En conclusión, este ejercicio demuestra cómo es posible desarrollar aplicaciones móviles de manera accesible, aun sin experiencia avanzada en programación, utilizando plataformas visuales como MIT App Inventor y entendiendo los fundamentos del ecosistema Android.